

Terraform with Azure

Compact Course — Day 2

Azure Infrastructure

Day 2 — Agenda

Working with Terraform and the Azure Cloud

- Creating a Terraform Service Principal in Azure
- Managed Identity and OIDC authentication
- Initializing the provider
- Using remote state with Storage Accounts and Blob Containers
- State operations: `state list`, `show`, `mv`, `rm`, `import`

Azure Virtual Network with Terraform

- Creating a Resource Group and Virtual Network
- VNet Peering, Route Tables, and Service Endpoints
- Testing with Virtual Machine instances

Azure Virtual Machine Instances with Terraform

- Creating Azure (Windows/Linux) Virtual Machine instances
- How attributes and blocks work (`source_image` , `size` , `os` , etc.)
- VM Extensions, Availability Zones, and boot diagnostics

Azure Network Security Groups

- Network Security Groups with dynamic blocks
- Application Security Groups
- Azure Network Interface

Monitoring with Azure Monitor

- Action groups, metric alerts, and diagnostic settings
- Log Analytics and KQL queries

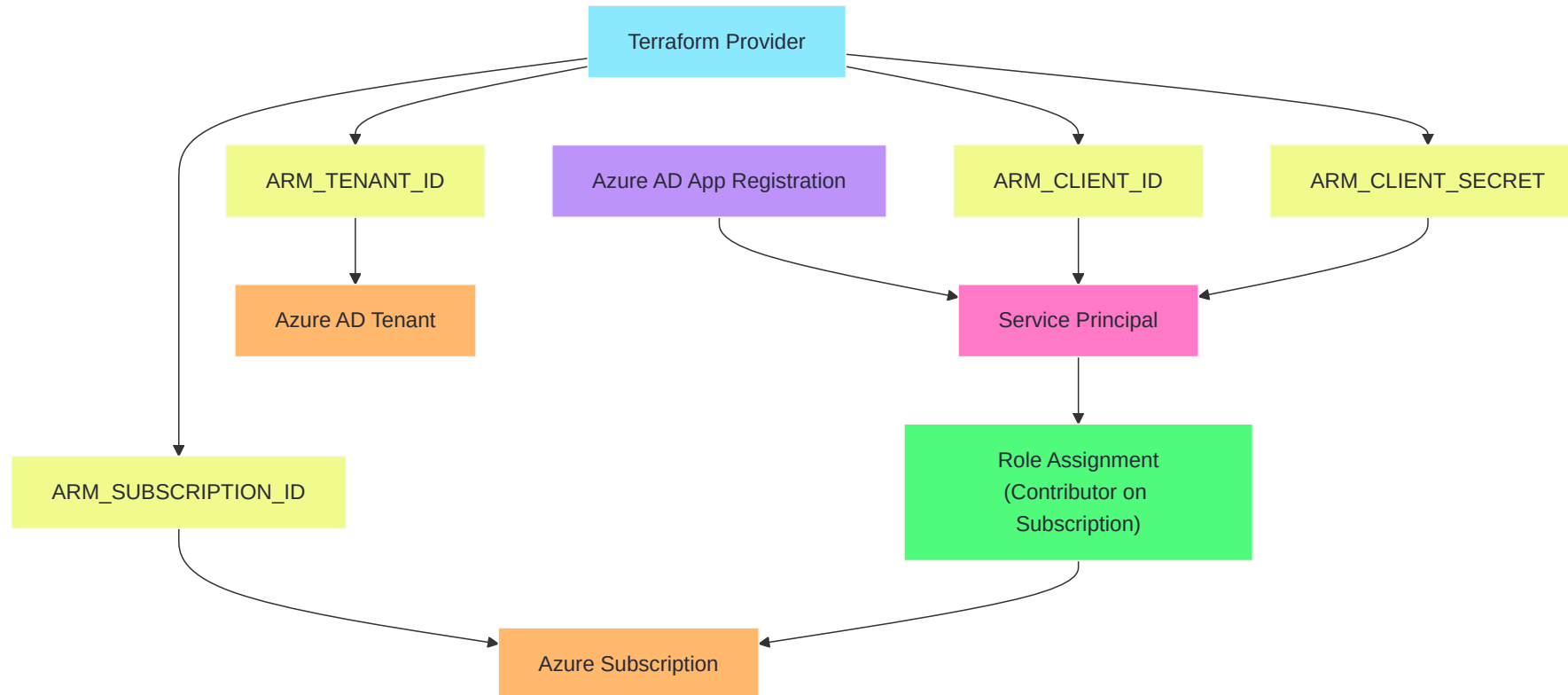
Part 1 — Working with Terraform and the Azure Cloud

Authentication Options for Azure

Method	Best for	How
Azure CLI	Local dev	<code>az login</code> — Terraform uses the session
Service Principal	CI/CD, teams	Client ID + Secret as env vars
Managed Identity	Azure-hosted runners	No credentials to manage
OIDC	GitHub Actions	Federated identity, no secrets

For this course we'll use **Service Principal** — the most common approach for real-world automation.

What is a Service Principal?



Creating a Service Principal

Step 1 — Create the Service Principal via Azure CLI

```
az ad sp create-for-rbac \  
  --name "sp-terraform-training" \  
  --role "Contributor" \  
  --scopes "/subscriptions/<SUBSCRIPTION_ID>"
```

Output

```
{
  "appId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "displayName": "sp-terraform-training",
  "password": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "tenant": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```

Save these somewhere safe — the `password` is shown **only once**.

Configuring Terraform to Use the Service Principal

Step 2 — Set environment variables

```
export ARM_CLIENT_ID="<appId>"  
export ARM_CLIENT_SECRET="<password>"  
export ARM_TENANT_ID="<tenant>"  
export ARM_SUBSCRIPTION_ID="<subscription_id>"
```

Step 3 — Configure the provider

```
terraform {  
  required_providers {  
    azurerm = {  
      source  = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}  
  
provider "azurerm" {  
  features {}  
}
```

When `ARM_*` env vars are set, the provider **picks them up automatically** — no secrets in code!

Managed Identity Authentication

For Terraform running **on an Azure VM or Azure DevOps agent**:

```
provider "azurerm" {  
  features {}  
  use_msi = true  # Use Managed Identity  
}
```

- **No credentials to manage** — Azure handles rotation
- The VM/container is assigned an identity with RBAC permissions
- Works with **system-assigned** or **user-assigned** managed identities

```
# Assign Contributor role to a VM's system-assigned identity
az vm identity assign --name my-build-vm --resource-group rg-cicd
az role assignment create \
  --assignee <principal-id> \
  --role Contributor \
  --scope /subscriptions/<sub-id>
```

OIDC / Federated Credentials (GitHub Actions)

No secrets needed — uses **OpenID Connect**:

```
provider "azurerm" {  
  features {}  
  use_oidc = true  
}
```

```
# GitHub Actions workflow  
- uses: azure/login@v2  
  with:  
    client-id: ${ secrets.AZURE_CLIENT_ID }  
    tenant-id: ${ secrets.AZURE_TENANT_ID }  
    subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }  
  
- run: terraform apply -auto-approve
```

Method	Secrets	Rotation	Best for
Service Principal	Client Secret	Manual (expires)	CI/CD, teams
Managed Identity	None	Azure-managed	Azure-hosted runners
OIDC	None	N/A (token-based)	GitHub Actions, GitLab

Initializing the Provider

```
$ terraform init
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/azurerm v4.14.0...
- Installed hashicorp/azurerm v4.14.0 (signed by HashiCorp)

```
Terraform has been successfully initialized!
```

What `terraform init` does

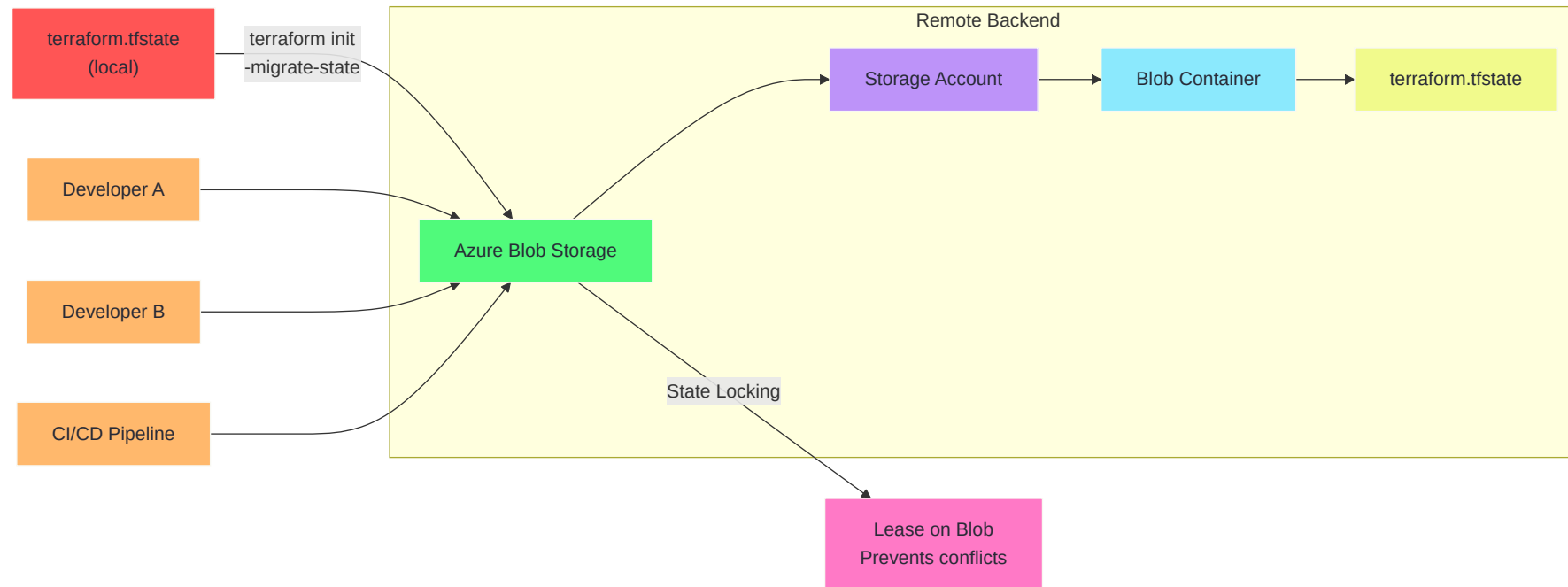
1. Downloads the `azurerm` provider plugin to `.terraform/providers/`
2. Creates `.terraform.lock.hcl` — pins the exact provider version
3. Initializes the backend (local or remote)

Run `terraform init` again whenever you add new providers or change the backend config.

Why Remote State?

Local state	Remote state
File on your machine	Stored in Azure Blob Storage
No locking — team conflicts	Automatic locking via blob lease
Lost if you delete the folder	Durable and backed up
No access control	RBAC on the storage account
Fine for learning	Required for teams & CI/CD

Remote State Architecture



Setting Up Remote State — Step 1: Create the Backend

```
# Create a resource group for the state
az group create --name rg-terraform-state --location westeurope

# Create a storage account (name must be globally unique)
az storage account create \
  --name stterraformstate42 \
  --resource-group rg-terraform-state \
  --sku Standard_LRS \
  --encryption-services blob

# Create a blob container
az storage container create \
  --name tfstate \
  --account-name stterraformstate42
```

This is the **one thing** you bootstrap outside of Terraform (or with a separate Terraform config).

Setting Up Remote State — Step 2: Configure the Backend

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "rg-terraform-state"  
    storage_account_name = "stterraformstate42"  
    container_name      = "tfstate"  
    key                 = "training.terraform.tfstate"  
  }  
  
  required_providers {  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}
```

```
# Migrate from local to remote
$ terraform init -migrate-state
Acquiring state lock. This may take a few moments...
Successfully configured the backend "azurerm"!
```

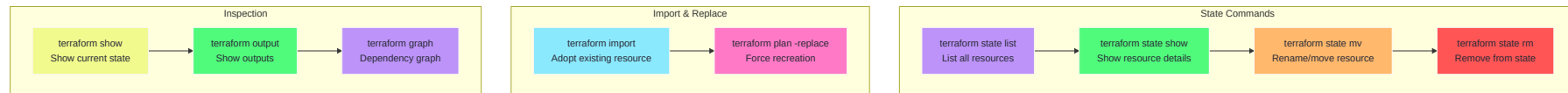
Remote State — Key Points

- **State locking:** Azure Blob uses a lease — only one `apply` at a time
- **State encryption:** enable encryption at rest on the storage account
- **State key:** use unique keys per project/environment (e.g., `dev.tfstate`, `prod.tfstate`)
- **Access control:** use RBAC to restrict who can read/write state

```
# Check who holds the lock (useful for debugging)
az storage blob show \
  --container-name tfstate \
  --name training.terraform.tfstate \
  --account-name stterraformstate42 \
  --query properties.lease
```

Never download and edit the remote state file manually. Use `terraform state` commands.

State Operations



terraform state list and show

```
# List all resources in state
$ terraform state list
azurerm_resource_group.main
azurerm_virtual_network.main
azurerm_subnet.subnets["web"]
azurerm_subnet.subnets["app"]

# Show details of a specific resource
$ terraform state show azurerm_resource_group.main
# azurerm_resource_group.main:
resource "azurerm_resource_group" "main" {
  id          = "/subscriptions/.../rg-dev-networking"
  location    = "westeurope"
  name        = "rg-dev-networking"
  tags        = {
    "environment" = "dev"
    "managed_by"  = "terraform"
  }
}
```

terraform state mv — Rename Without Destroy

```
# Rename a resource in state (prevents destroy/create cycle)
$ terraform state mv \
    azurerm_virtual_network.main \
    azurerm_virtual_network.primary
```

```
Move "azurerm_virtual_network.main" to "azurerm_virtual_network.primary"
Successfully moved 1 object(s).
```

Common use cases

- Renaming a resource in your `.tf` code
- Moving a resource into a module (`state mv azurerm_x.y module.z.azurerm_x.y`)
- Splitting a mono-config into modules

For new code, prefer the `moved` block (declarative) over `state mv` (imperative).

terraform state rm and import

Remove from state (stop managing, don't destroy)

```
$ terraform state rm azurerm_resource_group.legacy  
Removed azurerm_resource_group.legacy  
Successfully removed 1 resource instance(s).
```

Import existing resources into Terraform

```
# Step 1: Write the resource block in your .tf file
# Step 2: Import the existing Azure resource
$ terraform import azurerm_resource_group.existing \
    "/subscriptions/<sub>/resourceGroups/rg-existing"

azurerm_resource_group.existing: Importing...
azurerm_resource_group.existing: Import complete! [id=...]
```

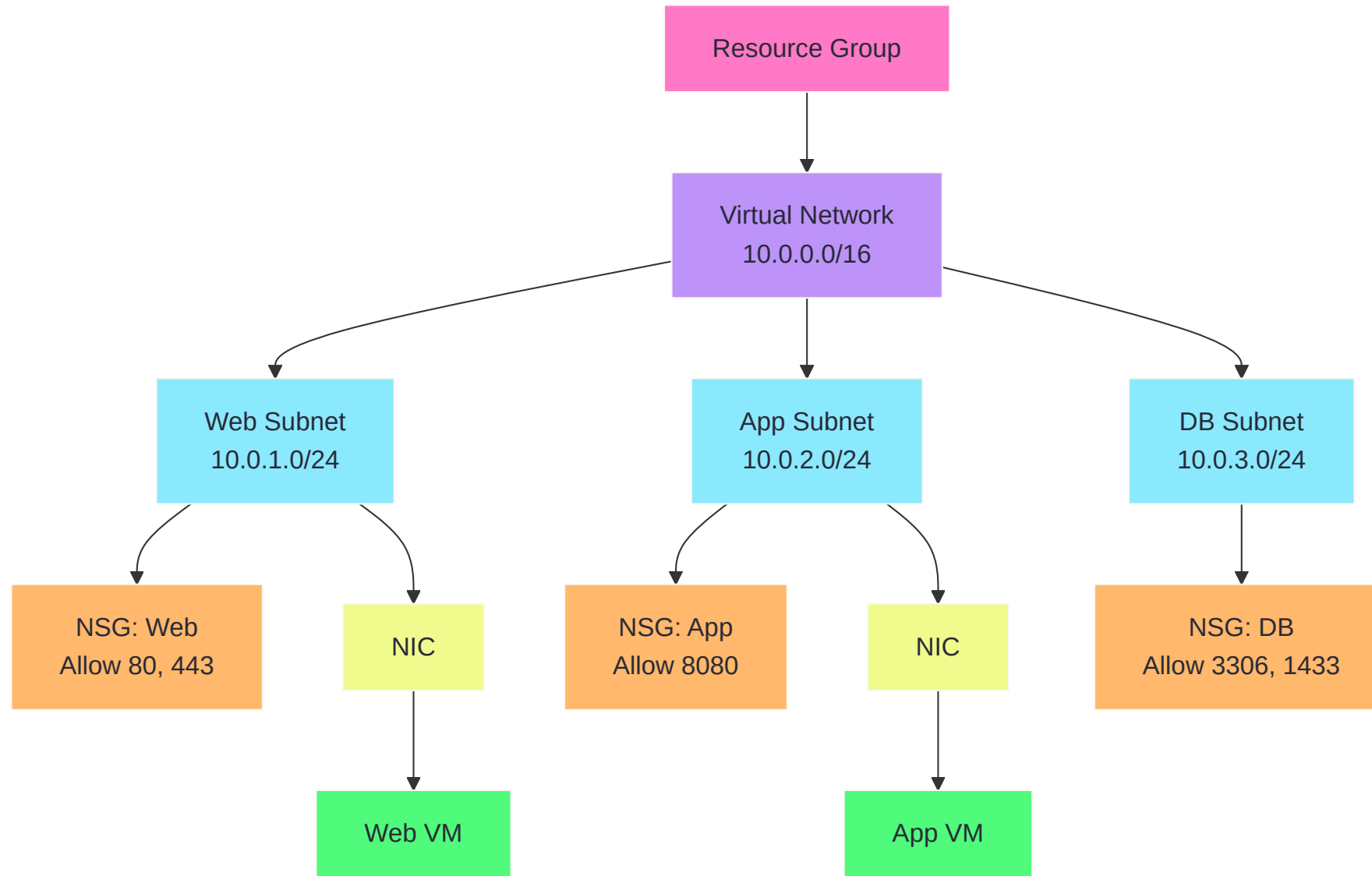
Import block (Terraform 1.5+)

```
import {
  to = azurerm_resource_group.existing
  id = "/subscriptions/<sub>/resourceGroups/rg-existing"
}
```

`import` is essential for adopting pre-existing Azure resources into Terraform management.

Part 2 — Azure Virtual Network with Terraform

Azure Networking — The Big Picture



Azure Networking — Key Concepts

Resource	Purpose
Resource Group	Logical container for all related resources
Virtual Network (VNet)	Isolated network in Azure with an address space
Subnet	Segment of a VNet with its own CIDR range
Network Security Group (NSG)	Firewall rules at the subnet or NIC level
Network Interface (NIC)	Connects a VM to a subnet
Public IP	Optional — exposes a resource to the internet

The hierarchy: **Resource Group** → **VNet** → **Subnet** → **NIC** → **VM**

Creating a Resource Group

```
variable "location" {  
  type      = string  
  default   = "westeurope"  
}  
  
variable "environment" {  
  type      = string  
  default   = "dev"  
}  
  
resource "azurerm_resource_group" "main" {  
  name      = "rg-${var.environment}-networking"  
  location  = var.location  
  
  tags = {  
    environment = var.environment  
    managed_by  = "terraform"  
  }  
}
```

Creating a Virtual Network

```
resource "azurerm_virtual_network" "main" {  
  name = "vnet-${var.environment}-main"  
  resource_group_name = azurerm_resource_group.main.name  
  location = azurerm_resource_group.main.location  
  address_space = ["10.0.0.0/16"]  
  
  tags = azurerm_resource_group.main.tags  
}
```

Key attributes

Attribute	Description
address_space	List of CIDR blocks for the VNet (usually one /16)
dns_servers	Custom DNS servers (optional, defaults to Azure DNS)
resource_group_name	Implicit dependency on the resource group

Creating Subnets

```
variable "subnets" {
  type = map(object({
    address_prefix = string
  }))
  default = {
    web = { address_prefix = "10.0.1.0/24" }
    app = { address_prefix = "10.0.2.0/24" }
    db  = { address_prefix = "10.0.3.0/24" }
  }
}

resource "azurerm_subnet" "subnets" {
  for_each = var.subnets

  name                = "snet-${var.environment}-${each.key}"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [each.value.address_prefix]
}
```

Subnets — **for_each** vs **count**

```
# With for_each – resources keyed by name
azurerm_subnet.subnets["web"]      → snet-dev-web
azurerm_subnet.subnets["app"]      → snet-dev-app
azurerm_subnet.subnets["db"]       → snet-dev-db

# With count – resources keyed by index
azurerm_subnet.subnets[0]          → snet-dev-0
azurerm_subnet.subnets[1]          → snet-dev-1
azurerm_subnet.subnets[2]          → snet-dev-2
```

	<code>count</code>	<code>for_each</code>
Key	Index (0, 1, 2)	Map key ("web", "app")
Add/remove	Shifts indexes → destroys/recreates	Only affects the specific key
Best for	Identical copies	Unique, named instances

Rule of thumb: use `for_each` for anything that has a meaningful name.

Testing the VNet with a VM

```
resource "azurerm_network_interface" "test" {
  name                = "nic-test-vm"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  ip_configuration {
    name                        = "internal"
    subnet_id                 = azurerm_subnet.subnets["web"].id
    private_ip_address_allocation = "Dynamic"
  }
}

resource "azurerm_linux_virtual_machine" "test" {
  name                = "vm-test"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  size                = "Standard_B1s"
  admin_username      = "adminuser"
  admin_password      = "P@ssw0rd1234!"
  disable_password_authentication = false
  network_interface_ids = [azurerm_network_interface.test.id]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}
```

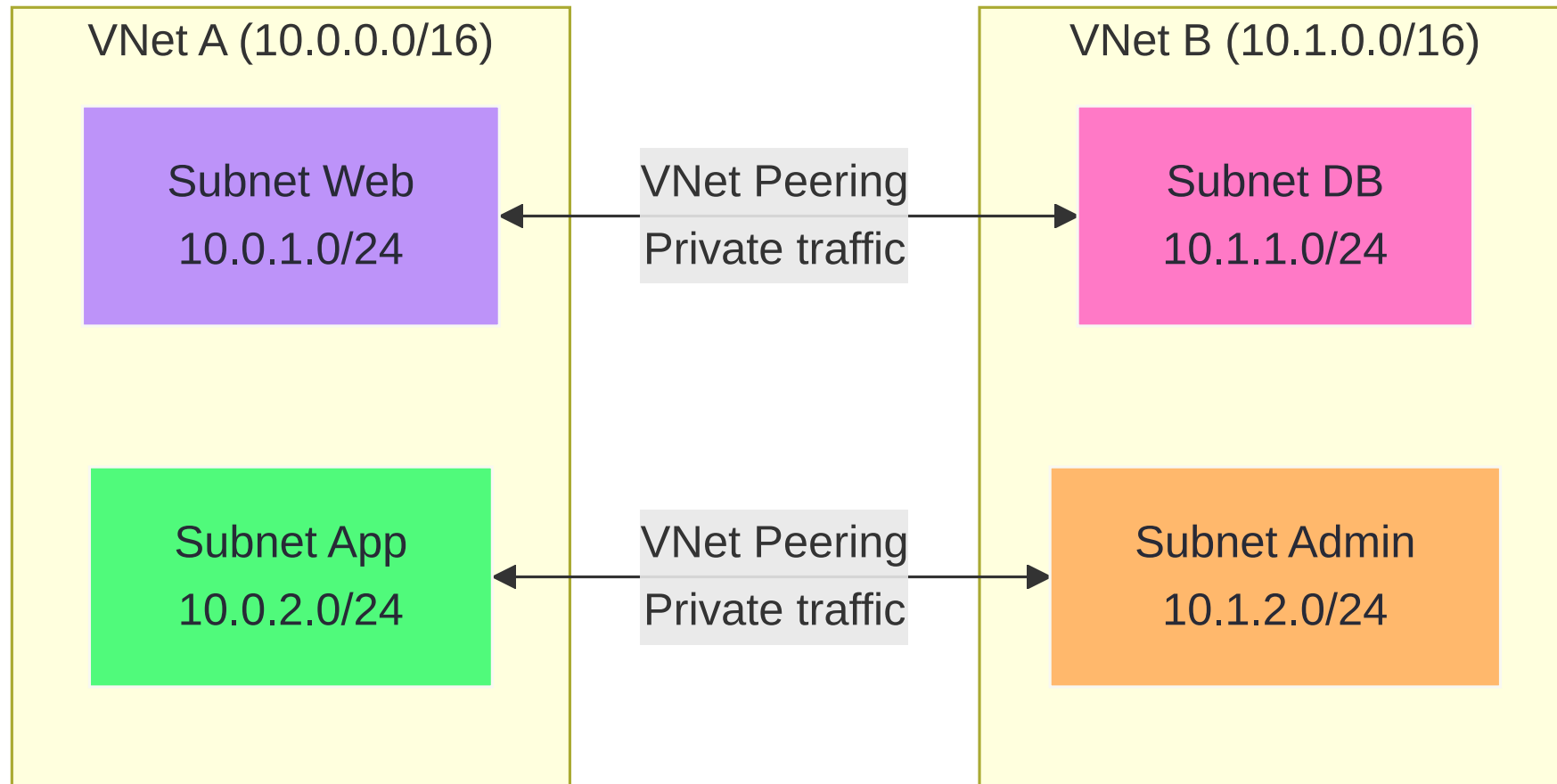
Verifying the Network

```
# Apply the config
$ terraform apply
Plan: 6 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 6 added

# Check via Azure CLI
$ az network vnet show \
    --resource-group rg-dev-networking \
    --name vnet-dev-main \
    --query '{addressSpace: addressSpace, subnets: subnets[].name}' \
    --output json

{
  "addressSpace": { "addressPrefixes": ["10.0.0.0/16"] },
  "subnets": ["snet-dev-web", "snet-dev-app", "snet-dev-db"]
}
```

VNet Peering — Connecting Virtual Networks



VNet Peering in Terraform

```
# Peering from VNet A → VNet B
resource "azurerm_virtual_network_peering" "a_to_b" {
  name                        = "peer-a-to-b"
  resource_group_name        = azurerm_resource_group.main.name
  virtual_network_name       = azurerm_virtual_network.a.name
  remote_virtual_network_id  = azurerm_virtual_network.b.id
  allow_forwarded_traffic    = true
}

# Peering from VNet B → VNet A (peering must be bidirectional)
resource "azurerm_virtual_network_peering" "b_to_a" {
  name                        = "peer-b-to-a"
  resource_group_name        = azurerm_resource_group.main.name
  virtual_network_name       = azurerm_virtual_network.b.name
  remote_virtual_network_id  = azurerm_virtual_network.a.id
  allow_forwarded_traffic    = true
}
```

Key points

- Peering is **non-transitive**: if A peers with B and B peers with C, A **cannot** reach C
- Address spaces **must not overlap**
- Peering can span regions (global VNet peering) and subscriptions

Route Tables (User-Defined Routes)

```
resource "azurerm_route_table" "main" {
  name                       = "rt-${var.environment}-main"
  resource_group_name       = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location

  route {
    name                     = "to-firewall"
    address_prefix           = "0.0.0.0/0"
    next_hop_type            = "VirtualAppliance"
    next_hop_in_ip_address = "10.0.0.4" # Azure Firewall IP
  }

  route {
    name                     = "to-vnet"
    address_prefix           = "10.0.0.0/16"
    next_hop_type            = "VnetLocal"
  }
}

resource "azurerm_subnet_route_table_association" "web" {
  subnet_id      = azurerm_subnet.subnets["web"].id
  route_table_id = azurerm_route_table.main.id
}
```

Service Endpoints

```
resource "azurerm_subnet" "app" {
  name                        = "snet-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  virtual_network_name       = azurerm_virtual_network.main.name
  address_prefixes           = ["10.0.2.0/24"]

  service_endpoints = [
    "Microsoft.Storage",
    "Microsoft.Sql",
    "Microsoft.KeyVault",
  ]
}
```

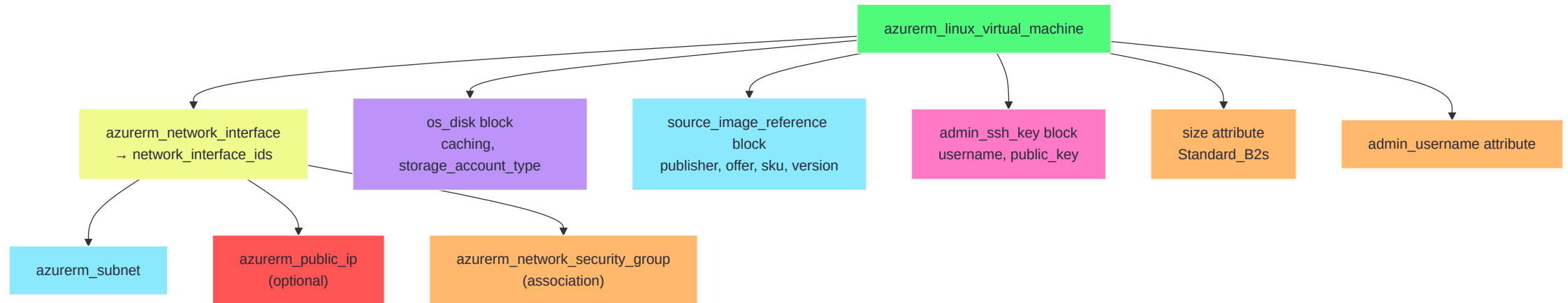
What service endpoints do

- Traffic to Azure PaaS services stays on the **Azure backbone** (not public internet)
- You can then restrict the PaaS resource to only accept traffic from that subnet
- **No extra cost**, no additional resources — just a subnet property

Supported services	Endpoint
Storage	<code>Microsoft.Storage</code>
SQL	<code>Microsoft.Sql</code>
Key Vault	<code>Microsoft.KeyVault</code>
Cosmos DB	<code>Microsoft.AzureCosmosDB</code>
Event Hub	<code>Microsoft.EventHub</code>

Part 3 — Azure Virtual Machine Instances

VM Resource Anatomy



VM Dependencies — What You Need First

Before creating a VM, these resources must exist:

1. **Resource Group** — container for everything
2. **Virtual Network** — the network your VM lives in
3. **Subnet** — the specific network segment
4. **Network Interface** — connects the VM to the subnet
5. *(Optional)* **Public IP** — if the VM needs internet access
6. *(Optional)* **NSG** — firewall rules

Terraform handles the ordering automatically through implicit dependencies (resource references).

Creating a Linux VM

```
resource "azurerm_linux_virtual_machine" "web" {
  name                        = "vm-${var.environment}-web"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  size                       = var.vm_size # e.g., "Standard_B2s"
  admin_username             = "adminuser"
  disable_password_authentication = true
  network_interface_ids      = [azurerm_network_interface.web.id]

  admin_ssh_key {
    username   = "adminuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Premium_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}
```

Understanding Blocks vs. Attributes

Attributes — single values

```
name      = "vm-dev-web"      # string
size      = "Standard_B2s"    # string
location  = "westeurope"      # string
```

Blocks — nested configuration objects

```
os_disk {                                # block
  caching          = "ReadWrite"
  storage_account_type = "Premium_LRS"
}

source_image_reference {                # block
  publisher = "Canonical"
  offer      = "0001-com-ubuntu-server-jammy"
  sku        = "22_04-lts"
  version    = "latest"
}

admin_ssh_key {                          # block (repeatable)
  username    = "adminuser"
  public_key  = file("~/ssh/id_rsa.pub")
}
```

Common `source_image_reference` Values

OS	Publisher	Offer	SKU
Ubuntu 22.04	Canonical	0001-com-ubuntu-server-jammy	22_04-lts
Ubuntu 24.04	Canonical	ubuntu-24_04-lts	server
Debian 12	Debian	debian-12	12
RHEL 9	RedHat	RHEL	9_2
Windows Server 2022	MicrosoftWindowsServer	WindowsServer	2022-Datacenter

Find available images

```
az vm image list --publisher Canonical --offer ubuntu --sku 22_04-lts \  
--location westeurope --output table --all
```

Creating a Windows VM

```
resource "azurerm_windows_virtual_machine" "app" {
  name                        = "vm-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  size                       = "Standard_B2ms"
  admin_username             = "adminuser"
  admin_password             = var.admin_password # sensitive variable
  network_interface_ids     = [azurerm_network_interface.app.id]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Premium_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-Datacenter"
    version   = "latest"
  }
}
```

VM Sizes — Choosing the Right One

Series	Use case	Example
B	Burstable, dev/test	Standard_B1s , Standard_B2s
D	General purpose	Standard_D2s_v5 , Standard_D4s_v5
E	Memory-optimized	Standard_E2s_v5
F	Compute-optimized	Standard_F2s_v2
N	GPU	Standard_NC6s_v3

Parameterize it!

```
variable "vm_size" {  
  type      = string  
  default   = "Standard_B2s"  
}
```

```
# dev.tfvars  
vm_size = "Standard_B1s"
```

```
# prod.tfvars  
vm_size = "Standard_D2s_v5"
```

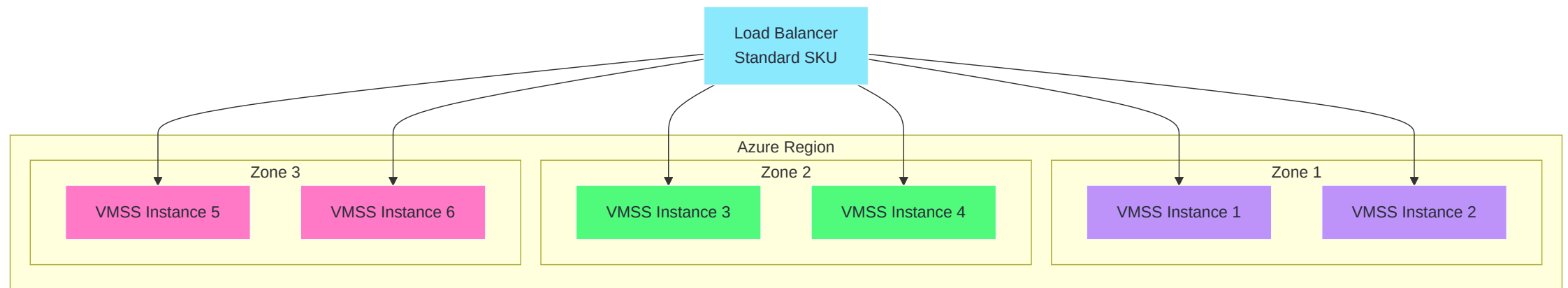
What Happens When You Change VM Attributes?

Change	Effect
<code>tags</code>	In-place update 
<code>size</code>	In-place update (VM restarts) 
<code>os_disk.storage_account_type</code>	Replacement  — VM destroyed & recreated
<code>source_image_reference</code>	Replacement  — VM destroyed & recreated
<code>admin_username</code>	Replacement 
<code>name</code>	Replacement 

Always run `terraform plan` before `apply` to see if a change triggers replacement!

```
# azurerm_linux_virtual_machine.web must be replaced
-/+ resource "azurerm_linux_virtual_machine" "web" {
    ~ name = "vm-dev-web" -> "vm-dev-web-2" # forces replacement
}
```

Availability Zones



Availability Zones in Terraform

```
resource "azurerm_linux_virtual_machine" "web" {  
  name                        = "vm-${var.environment}-web"  
  resource_group_name        = azurerm_resource_group.main.name  
  location                   = azurerm_resource_group.main.location  
  size                       = "Standard_D2s_v5"  
  zone                       = "1"    # Pin to Availability Zone 1  
  # ...  
}
```

Key points

Concept	Description
Availability Zone	Physically separate datacenter within a region
Zone numbers	"1" , "2" , "3" (not all regions have 3)
Zone-redundant	Spread across zones for HA (LB, VMSS)
Zonal	Pinned to one zone (single VM)

Always deploy production workloads across **multiple zones** using a load balancer or VMSS.

VM Extensions — Post-Deployment Configuration

```
resource "azurerm_virtual_machine_extension" "monitor" {  
  name                        = "AzureMonitorAgent"  
  virtual_machine_id         = azurerm_linux_virtual_machine.web.id  
  publisher                  = "Microsoft.Azure.Monitor"  
  type                       = "AzureMonitorLinuxAgent"  
  type_handler_version       = "1.0"  
  auto_upgrade_minor_version = true  
}
```

Common extensions

Extension	Publisher	Purpose
Azure Monitor Agent	<code>Microsoft.Azure.Monitor</code>	Send metrics/logs
Custom Script	<code>Microsoft.Azure.Extensions</code>	Run a bash/PS script
AAD SSH Login	<code>Microsoft.Azure.ActiveDirectory</code>	Azure AD auth for SSH
Dependency Agent	<code>Microsoft.Azure.Monitoring.DependencyAgent</code>	Service Map

Use `custom_data` for initial bootstrapping; use **extensions** for ongoing agent management.

Boot Diagnostics

```
resource "azurerm_linux_virtual_machine" "web" {  
  name = "vm-${var.environment}-web"  
  # ... (rest of VM config)  
  
  boot_diagnostics {  
    storage_account_uri = azurerm_storage_account.diag.primary_blob_endpoint  
  }  
}
```

What it provides

- **Serial console** output — see boot logs without SSH
- **Screenshot** — see the VM's display (useful for Windows stuck at login)
- Stored in a **storage account** you specify

```
# Or use managed boot diagnostics (simpler, no storage account needed)
boot_diagnostics {} # empty block = Azure-managed storage
```

Enable boot diagnostics on all production VMs — it's the first thing you check when a VM won't start.

Tagging Strategy

```
locals {
  common_tags = {
    environment = var.environment
    project     = var.project_name
    managed_by  = "terraform"
    cost_center = var.cost_center
    owner       = var.team_email
    created_date = formatdate("YYYY-MM-DD", timestamp())
  }
}

resource "azurerm_linux_virtual_machine" "web" {
  # ...
  tags = merge(local.common_tags, {
    role = "web-server"
    tier  = "frontend"
  })
}
```

Part 4 — Azure Network Security Groups

What is a Network Security Group?

- A **firewall** that filters traffic to and from Azure resources
- Contains **security rules** with priorities (100–4096)
- Can be associated with a **subnet** or a **network interface**
- Rules are evaluated in **priority order** (lowest number = highest priority)

Field	Description
priority	100–4096, lower = evaluated first
direction	Inbound or Outbound
access	Allow or Deny
protocol	Tcp , Udp , Icmp , or *
source/destination_port_range	Port or range (80 , 1024-65535 , *)
source/destination_address_prefix	CIDR, tag, or *

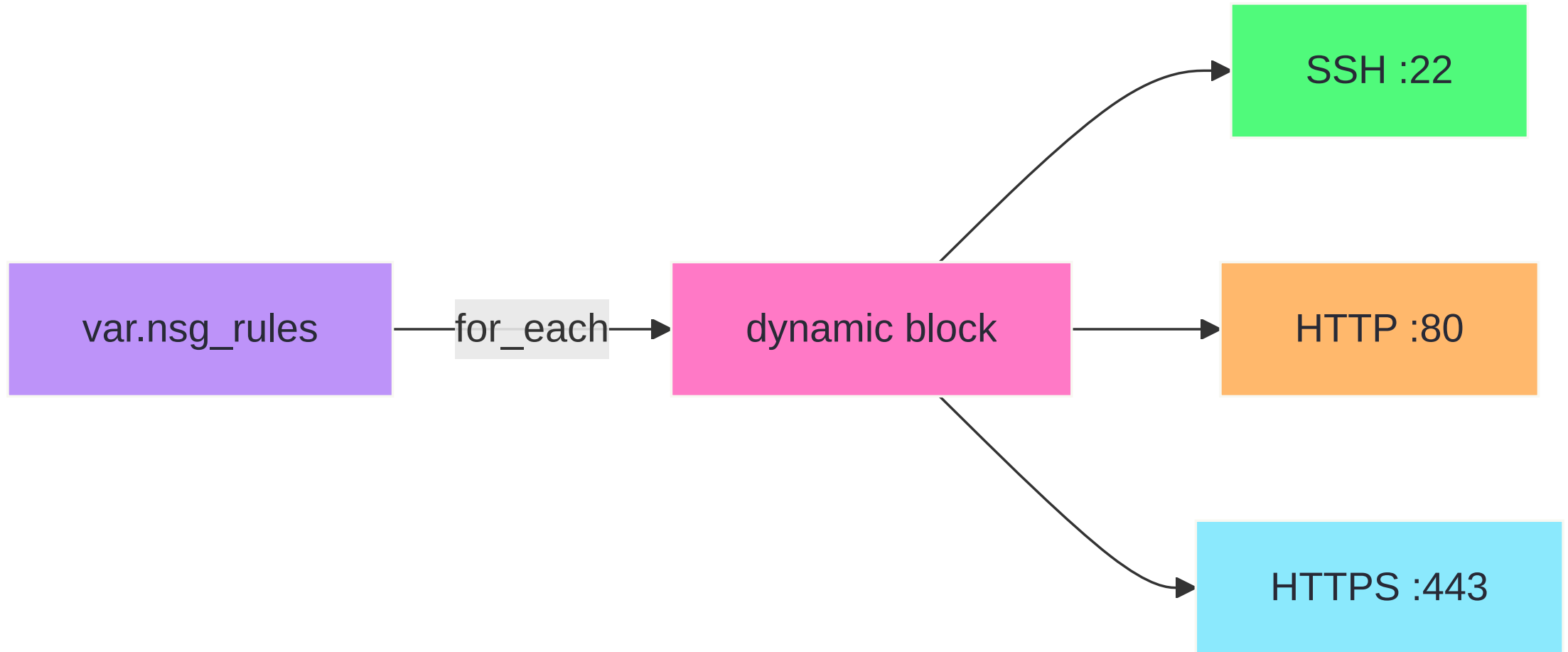
Creating an NSG — The Manual Way

```
resource "azurerm_network_security_group" "web" {
  name                        = "nsg-${var.environment}-web"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location

  security_rule {
    name                        = "AllowSSH"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name                        = "AllowHTTP"
    priority                   = 200
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "80"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}
```

Dynamic Blocks — The Concept



NSG with Dynamic Blocks

```
variable "nsg_rules" {
  type = list(object({
    name                = string
    priority            = number
    direction           = string
    access              = string
    protocol            = string
    destination_port_range = string
  }))
  default = [
    { name = "AllowSSH",    priority = 100, direction = "Inbound",
      access = "Allow", protocol = "Tcp", destination_port_range = "22" },
    { name = "AllowHTTP",  priority = 200, direction = "Inbound",
      access = "Allow", protocol = "Tcp", destination_port_range = "80" },
    { name = "AllowHTTPS", priority = 300, direction = "Inbound",
      access = "Allow", protocol = "Tcp", destination_port_range = "443" },
  ]
}
```

NSG with Dynamic Blocks — The Resource

```
resource "azurerm_network_security_group" "web" {
  name                        = "nsg-${var.environment}-web"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location

  dynamic "security_rule" {
    for_each = var.nsg_rules
    content {
      name                        = security_rule.value.name
      priority                   = security_rule.value.priority
      direction                  = security_rule.value.direction
      access                     = security_rule.value.access
      protocol                   = security_rule.value.protocol
      source_port_range          = "*"
      destination_port_range    = security_rule.value.destination_port_range
      source_address_prefix      = "*"
      destination_address_prefix = "*"
    }
  }
}
```

Dynamic Block Syntax — Deep Dive

```
dynamic "<BLOCK_LABEL>" {  
  for_each = <COLLECTION>          # list, map, or set to iterate  
  iterator = <NAME>                # optional – rename the iterator  
  content {  
    # use <BLOCK_LABEL>.value or <NAME>.value  
    # use <BLOCK_LABEL>.key or <NAME>.key  
  }  
}
```

Key points

- `for_each` accepts `list`, `map`, or `set`
- Inside `content`, use `<label>.value` for the current element
- Use `<label>.key` for the index (list) or key (map)
- `iterator` is optional — only needed if the default name conflicts

Associating NSG with a Subnet

```
resource "azurerm_subnet_network_security_group_association" "web" {  
  subnet_id          = azurerm_subnet.subnets["web"].id  
  network_security_group_id = azurerm_network_security_group.web.id  
}
```

Or associate with a **network interface** instead:

```
resource "azurerm_network_interface_security_group_association" "web" {  
  network_interface_id      = azurerm_network_interface.web.id  
  network_security_group_id = azurerm_network_security_group.web.id  
}
```

Scope	When to use
Subnet	Apply the same rules to all resources in a subnet
NIC	Apply specific rules to a single VM

You can combine both — rules from both NSGs are merged.

Azure Network Interface — Full Example

```
resource "azurerm_public_ip" "web" {
  name                        = "pip-${var.environment}-web"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  allocation_method          = "Static"
  sku                        = "Standard"
}

resource "azurerm_network_interface" "web" {
  name                        = "nic-${var.environment}-web"
  location                   = azurerm_resource_group.main.location
  resource_group_name        = azurerm_resource_group.main.name

  ip_configuration {
    name                        = "primary"
    subnet_id                  = azurerm_subnet.subnets["web"].id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id        = azurerm_public_ip.web.id
  }
}
```

Application Security Groups (ASGs)

```
resource "azurerm_application_security_group" "web_servers" {  
  name = "asg-web-servers"  
  resource_group_name = azurerm_resource_group.main.name  
  location = azurerm_resource_group.main.location  
}  
  
# Associate a NIC with the ASG  
resource "azurerm_network_interface_application_security_group_association" "web" {  
  network_interface_id = azurerm_network_interface.web.id  
  application_security_group_id = azurerm_application_security_group.web_servers.id  
}
```

Use ASGs in NSG rules

```
security_rule {  
  name = "AllowHTTPFromApp"  
  priority = 200  
  direction = "Inbound"  
  access = "Allow"  
  protocol = "Tcp"  
  destination_port_range = "80"  
  source_application_security_group_ids = [azurerm_application_security_group.app_servers.id]  
  destination_application_security_group_ids = [azurerm_application_security_group.web_servers.id]  
}
```

ASGs let you write NSG rules based on **roles** instead of IP addresses — much easier to maintain.

NSG Flow Logs

```
resource "azurerm_network_watcher_flow_log" "nsg" {
  network_watcher_name = "NetworkWatcher_westeurope"
  resource_group_name  = "NetworkWatcherRG"
  name                 = "flowlog-nsg-web"

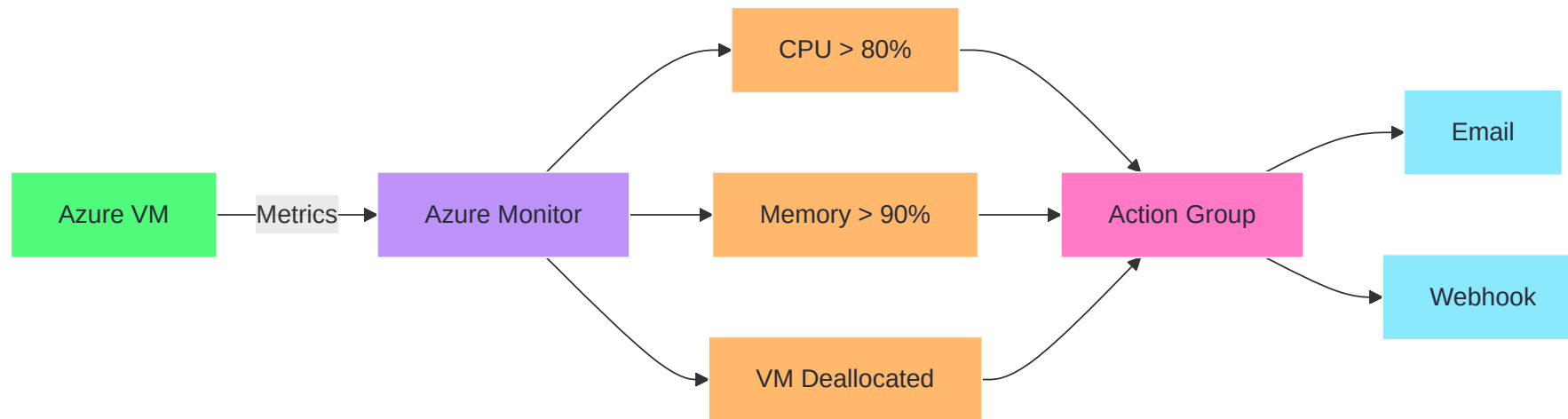
  network_security_group_id = azurerm_network_security_group.web.id
  storage_account_id       = azurerm_storage_account.logs.id
  enabled                   = true
  version                   = 2

  retention_policy {
    enabled = true
    days    = 30
  }

  traffic_analytics {
    enabled           = true
    workspace_id      = azurerm_log_analytics_workspace.main.workspace_id
    workspace_region  = azurerm_log_analytics_workspace.main.location
    workspace_resource_id = azurerm_log_analytics_workspace.main.id
    interval_in_minutes = 10
  }
}
```

Part 5 — Monitoring with Azure Monitor

Azure Monitor Overview



Azure Monitor — Key Components

Component	Purpose	Terraform resource
Action Group	Defines who to notify and how	<code>azurerm_monitor_action_group</code>
Metric Alert	Triggers on metric thresholds	<code>azurerm_monitor_metric_alert</code>
Activity Log Alert	Triggers on control plane events	<code>azurerm_monitor_activity_log_alert</code>
Diagnostic Settings	Sends logs/metrics to a destination	<code>azurerm_monitor_diagnostic_setting</code>
Log Analytics Workspace	Central log storage & query engine	<code>azurerm_log_analytics_workspace</code>

Creating an Action Group

```
resource "azurerm_monitor_action_group" "ops_team" {
  name                        = "ag-ops-team"
  resource_group_name        = azurerm_resource_group.main.name
  short_name                  = "OpsTeam"

  email_receiver {
    name            = "ops-email"
    email_address   = "ops-team@example.com"
  }

  webhook_receiver {
    name            = "slack-webhook"
    service_uri     = "https://hooks.slack.com/services/XXX/YYY/ZZZ"
  }
}
```

Action groups define **who gets notified** — email, SMS, webhook, Azure Function, Logic App, etc.

Creating a Metric Alert

```
resource "azurerm_monitor_metric_alert" "cpu_high" {
  name = "alert-cpu-high"
  resource_group_name = azurerm_resource_group.main.name
  scopes = [azurerm_linux_virtual_machine.web.id]
  description = "Alert when CPU exceeds 80% for 5 minutes"
  severity = 2
  frequency = "PT1M"
  window_size = "PT5M"

  criteria {
    metric_namespace = "Microsoft.Compute/virtualMachines"
    metric_name = "Percentage CPU"
    aggregation = "Average"
    operator = "GreaterThan"
    threshold = 80
  }

  action {
    action_group_id = azurerm_monitor_action_group.ops_team.id
  }
}
```

Metric Alert — Key Parameters

Parameter	Description
<code>scopes</code>	List of resource IDs to monitor
<code>severity</code>	0 (Critical) to 4 (Informational)
<code>frequency</code>	How often to evaluate: <code>PT1M</code> , <code>PT5M</code> , <code>PT15M</code>
<code>window_size</code>	Time window for aggregation: <code>PT5M</code> , <code>PT1H</code>
<code>criteria.aggregation</code>	<code>Average</code> , <code>Minimum</code> , <code>Maximum</code> , <code>Total</code> , <code>Count</code>
<code>criteria.operator</code>	<code>GreaterThan</code> , <code>LessThan</code> , <code>Equals</code> , etc.

Common metrics for VMs

Metric	Namespace
Percentage CPU	Microsoft.Compute/virtualMachines
Available Memory Bytes	Microsoft.Compute/virtualMachines
Disk Read Bytes	Microsoft.Compute/virtualMachines
Network In Total	Microsoft.Compute/virtualMachines

Diagnostic Settings — Send Logs to Log Analytics

```
resource "azurerm_log_analytics_workspace" "main" {
  name                        = "log-${var.environment}-main"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  sku                        = "PerGB2018"
  retention_in_days          = 30
}

resource "azurerm_monitor_diagnostic_setting" "vnet" {
  name                        = "diag-vnet"
  target_resource_id          = azurerm_virtual_network.main.id
  log_analytics_workspace_id = azurerm_log_analytics_workspace.main.id

  enabled_log {
    category = "VMProtectionAlerts"
  }

  metric {
    category = "AllMetrics"
  }
}
```

Log Analytics — KQL Queries

After setting up diagnostic settings, query your logs with **Kusto Query Language (KQL)**:

```
-- Find all failed NSG flow logs (blocked traffic)
AzureNetworkAnalytics_CL
| where FlowStatus_s == "D"
| summarize count() by SrcIP_s, DestPort_d
| order by count_ desc
| take 20
```

```
-- VM CPU over time
Perf
| where ObjectName == "Processor" and CounterName == "% Processor Time"
| where Computer == "vm-dev-web"
| summarize avg(CounterValue) by bin(TimeGenerated, 5m)
| render timechart
```

You can run KQL queries in the Azure Portal under **Log Analytics workspaces > Logs**.

Activity Log Alerts

```
resource "azurerm_monitor_activity_log_alert" "rg_delete" {
  name                        = "alert-rg-delete"
  resource_group_name        = azurerm_resource_group.main.name
  scopes                     = ["/subscriptions/${data.azurerm_subscription.current.subscription_id}"]
  description                 = "Alert when a resource group is deleted"

  criteria {
    category      = "Administrative"
    operation_name = "Microsoft.Resources/subscriptions/resourceGroups/delete"
    level         = "Critical"
  }

  action {
    action_group_id = azurerm_monitor_action_group.ops_team.id
  }
}
```

Common activity log triggers

Operation	Category	Use case
Resource group delete	Administrative	Prevent accidental deletion
Role assignment change	Administrative	Security audit
Service health events	ServiceHealth	Outage notifications
Auto-scale events	Autoscale	Track scaling behavior

Day 2 — Recap

What We Covered Today

Azure Provider Setup

- Service Principal, Managed Identity, and OIDC authentication
- Remote state in Azure Blob Storage with locking
- State operations: `list` , `show` , `mv` , `rm` , `import`

Virtual Networks

- VNet, subnets with `for_each` , resource group naming conventions
- `for_each` vs `count` for named resources
- VNet Peering, Route Tables, and Service Endpoints

Virtual Machines

- Linux and Windows VMs, `source_image_reference`, blocks vs attributes
- VM sizing, availability zones, extensions, boot diagnostics, tagging
- In-place updates vs replacement

Network Security Groups

- NSG rules, dynamic blocks for DRY configuration
- Application Security Groups for role-based rules
- Subnet vs NIC association, NSG flow logs

Monitoring

- Action groups, metric alerts, activity log alerts, diagnostic settings
- Log Analytics, KQL queries

Exercises

See the `exercises/day-2/` folder for four hands-on labs:

1. **Service Principal and Remote State Setup**
2. **Virtual Network and Subnets**
3. **Virtual Machines with Attributes and Blocks**
4. **NSGs, Dynamic Blocks, and Monitoring**